

AIMLCZG549 - API-driven Cloud Native Solutions

Assignment I - Submission Report

Group Details

Group No: 11

Group Member Names with Contribution

BITS ID	Name	Contribution (Qualitative)	% Contribution (Quantitative)
2024AC05203	Kodhandan S	Data pipeline: ingestion, pre- processing, EDA, DataOps scheduler	100%
2024AC05046	Gowrishankar S	ML pipeline: model training, evaluation, MLOps tracking	100%
2024AC05045	Arunkumar K A	API access module, dashboard integration	100%
2024AC05219	Naveen R	Report documentation, screenshots, video demo	100%
2024AC05304	KUMARESH BABU A	Testing, deployment, README/setup instructions	100%

Project Overview

Business Problem: Telecom customer churn is a major revenue-loss driver for subscription-based telecom operators. This project builds a cloud-native Data Science / Machine Learning application that predicts which customers are likely to churn (cancel their subscription), so that the retention/marketing team can proactively target at-risk customers with offers.

Dataset: IBM/Kaggle "Telco Customer Churn" dataset

(<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>) - 7,043 customer records with 21 attributes (demographics, account information, subscribed services, and the Churn label).

Project architecture:

- **Data Pipeline:** ingestion -> pre-processing -> EDA, automated by a DataOps scheduler that re-runs every 2 minutes and logs every step.
- **ML Pipeline:** trains 2 algorithms (Logistic Regression, Random Forest) on a 70/30 split, evaluates them, and an MLOps layer (MLflow) tracks & logs metrics for every run.
- **API Access:** the built-in MLflow REST API is queried to surface experiment / run / model-deployment details (no custom API code).
- **Cloud Dashboard:** a Streamlit application ties all of the above together into one interactive UI, deployable to Streamlit Community Cloud.

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

 Overview  Data Pipeline  EDA  DataOps (Scheduler)  ML Pipeline & MLOps  API Acc

Total customers	Features	Churn rate	Data source
7,043	20	26.5%	Kaggle...

Sample records

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mu
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	No
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes
7	6713-OKOMC	Female	0	No	No	10	No	No
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes
9	6388-TABGU	Male	0	No	Yes	62	Yes	No

Fig 1. Dashboard - Overview tab showing dataset summary (7,043 customers, 20 features, 26.5% churn rate).

Sub-Objective 1: Design and Development of a Data Pipeline (Weight: 8 marks)

1.1 Business Understanding

The business problem addressed is customer churn prediction for a telecom operator. Reducing churn even by a small percentage has a large impact on recurring revenue, since acquiring a new customer costs significantly more than retaining an existing one. The goal is to identify customers at high risk of churning using their account, billing and subscribed-service attributes, so that retention campaigns can be targeted effectively.

1.2 Data Ingestion

Dataset source: Kaggle - "Telco Customer Churn" (blastchar/telco-customer-churn), 7,043 records x 21 columns. The ingestion module (src/data_pipeline/ingestion.py) loads the CSV from data/raw/; if the file is missing (e.g. on a fresh machine after unzipping this project), src/data_pipeline/download_data.py automatically downloads it from a public GitHub mirror of the same dataset, or prints manual Kaggle-download instructions if there is no internet access. This satisfies the requirement of a dataset with sufficient records for a meaningful experiment.

1.3 Data Pre-processing

Implemented in src/data_pipeline/preprocessing.py. Activities performed on every pipeline run:

- Summary statistics: count, mean, std, min/max, quartiles for every column (df.describe()).
- Missing-value check: TotalCharges has 11 blank-string values (0.16%) corresponding to brand-new customers with tenure = 0.
- Imputation: missing numeric values are imputed with the column median (TotalCharges median = 1397.47).
- Data types: displayed for every column (object / int64 / float64).
- Normalization: tenure, MonthlyCharges, TotalCharges and SeniorCitizen are Min-Max scaled into new *_norm columns in the [0, 1] range.

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

 Overview  **Data Pipeline**  EDA  DataOps (Scheduler)  ML Pipeline & MLOps  API Acc

Data Ingestion & Pre-processing

1.3 Summary statistics (raw data)

	count	unique	top	freq	mean	std	min	25%	50%	75%
customerID	7,043	7,043	7590-VHVEG	1	None	None	None	None	None	No
gender	7,043	2	Male	3,555	None	None	None	None	None	No
SeniorCitizen	7,043	None	None	None	0.1621	0.3686	0	0	0	
Partner	7,043	2	No	3,641	None	None	None	None	None	No
Dependents	7,043	2	No	4,933	None	None	None	None	None	No
tenure	7,043	None	None	None	12.3711	14.5595	0	9	29	
PhoneService	7,043	2	Yes	6,361	None	None	None	None	None	No
MultipleLines	7,043	3	No	3,390	None	None	None	None	None	No
InternetService	7,043	3	Fiber optic	3,096	None	None	None	None	None	No
OnlineSecurity	7,043	3	No	3,498	None	None	None	None	None	No

Fig 2. Summary statistics of the raw dataset.

Deploy ⋮

customerID	7,043	7,043	7,590-VHVEG	1	None	None	None	None	None	No
gender	7,043	2	Male	3,555	None	None	None	None	None	No
SeniorCitizen	7,043	None	None	None	0.1621	0.3686	0	0	0	
Partner	7,043	2	No	3,641	None	None	None	None	None	No
Dependents	7,043	2	No	4,933	None	None	None	None	None	No
tenure	7,043	None	None	None	12.3711	14.5595	0	9	29	
PhoneService	7,043	2	Yes	6,361	None	None	None	None	None	No
MultipleLines	7,043	3	No	3,390	None	None	None	None	None	No
InternetService	7,043	3	Fiber optic	3,096	None	None	None	None	None	No
OnlineSecurity	7,043	3	No	3,498	None	None	None	None	None	No

Missing values

	missing_count
TotalCharges	11

`TotalCharges` has blank strings for brand-new customers (`tenure = 0`); these are imputed with the column median.

Data types

	dtype
customerID	object
gender	object
SeniorCitizen	int64
Partner	object

Fig 3. Missing-value report and column data types.

customerID	object
gender	object
SeniorCitizen	int64
Partner	object
Dependents	object
tenure	int64
PhoneService	object
MultipleLines	object
InternetService	object
OnlineSecurity	object

Processed & normalized data (preview)

	tenure	MonthlyCharges	TotalCharges	tenure_norm	MonthlyCharges_norm	TotalCharges_norm
0	1	29.85	29.85	0.0139	0.1154	0.0139
1	34	56.95	1,889.5	0.4722	0.3851	0.3851
2	2	53.85	108.15	0.0278	0.3542	0.0278
3	45	42.3	1,840.75	0.625	0.2393	0.2393
4	2	70.7	151.65	0.0278	0.5219	0.0278
5	8	99.65	820.5	0.1111	0.81	0.1111
6	22	89.1	1,949.4	0.3056	0.705	0.3056
7	10	29.75	301.9	0.1389	0.1144	0.1144
8	28	104.8	3,046.05	0.3889	0.8612	0.3889
9	62	56.15	3,487.95	0.8611	0.3771	0.3771

Fig 4. Pre-processed data preview with Min-Max normalized columns.

1.4 Exploratory Data Analysis (EDA)

Implemented in `src/data_pipeline/eda.py`. Activities performed:

- Correlation coefficients: numeric features vs. the Churn target (e.g. tenure = -0.35, MonthlyCharges = +0.19).
- Correlation between categorical and target: categorical columns are label-encoded and correlated with Churn (Contract = -0.40, OnlineSecurity = -0.29, TechSupport = -0.28 are the strongest).

- Binning: tenure is binned into 5 groups (0-12, 13-24, 25-48, 49-60, 61-72 months) via tenure_group.
- Encoding: all categorical columns are Label-Encoded for downstream modelling and correlation analysis.
- Feature importance: a Random Forest is fit on the encoded features to rank importance - Contract, tenure and MonthlyCharges are the top predictors.
- Visualization: 7 charts are generated covering both univariate (churn distribution, tenure histogram, monthly-charges histogram) and bivariate (correlation heatmap, churn-rate by contract, tenure vs churn boxplot, feature importance) analysis.

Deploy ⋮

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

 Overview  Data Pipeline  EDA  DataOps (Scheduler)  ML Pipeline & MLOps  API Acc

Exploratory Data Analysis

Correlation with target (numeric features)

```

{
  "tenure" : -0.3522
  "MonthlyCharges" : 0.1934
  "TotalCharges" : -0.199
  "SeniorCitizen" : 0.1509
  "Churn" : 1
}

```

Correlation with target (categorical features)

```

{
  "Contract" : -0.3967
  "OnlineSecurity" : -0.2893
  "TechSupport" : -0.2825
  "OnlineBackup" : -0.1955
  "PaperlessBilling" : 0.1918
  "DeviceProtection" : -0.1781
  "Dependents" : -0.1642
  "Partner" : -0.1504
  "PaymentMethod" : 0.1071
  "InternetService" : -0.0473
  "StreamingMovies" : -0.0385
  "MultipleLines" : 0.038
}

```

Fig 5. Correlation of numeric and categorical features with Churn.

Deploy ⋮

```
"SeniorCitizen" : 0.1509
"Churn" : 1
}

"OnlineBackup" : -0.1955
"PaperlessBilling" : 0.1918
"DeviceProtection" : -0.1781
"Dependents" : -0.1642
"Partner" : -0.1504
"PaymentMethod" : 0.1071
"InternetService" : -0.0473
"StreamingMovies" : -0.0385
"MultipleLines" : 0.038
"StreamingTV" : -0.0366
"PhoneService" : 0.0119
"gender" : -0.0086
}
```

Top feature importances (Random Forest)

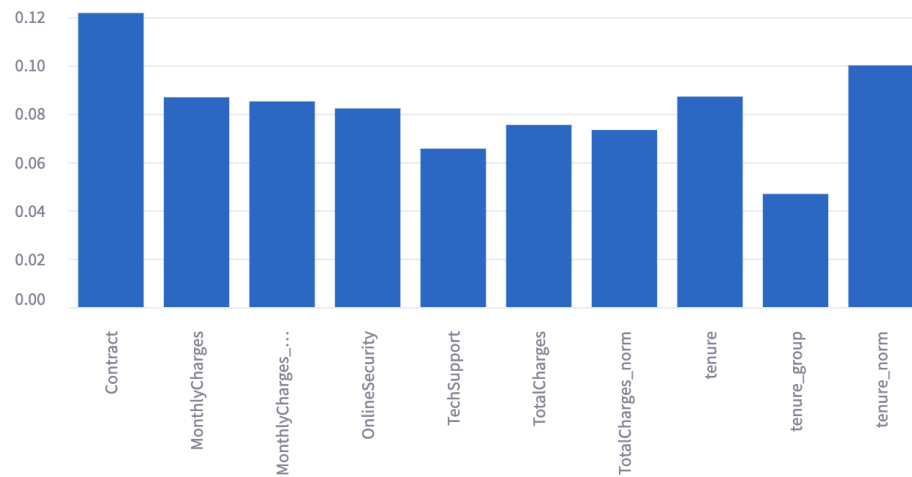
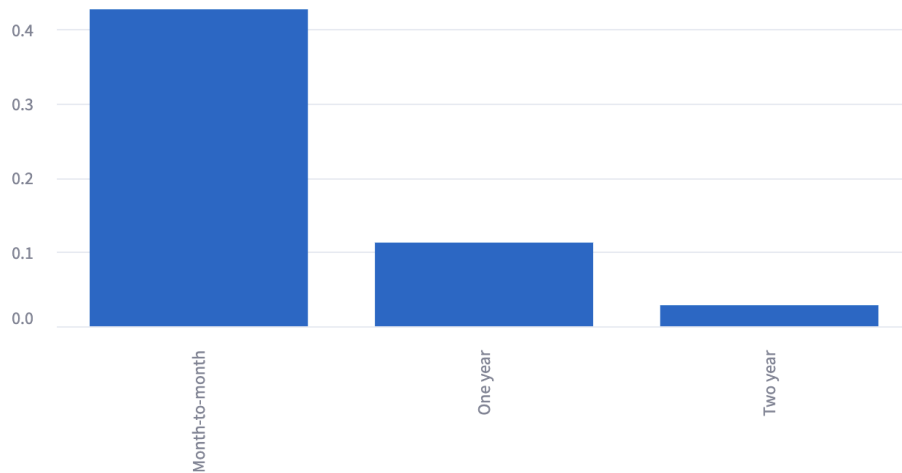


Fig 6. Random-Forest based feature importance ranking.



Visualizations (univariate + bivariate)

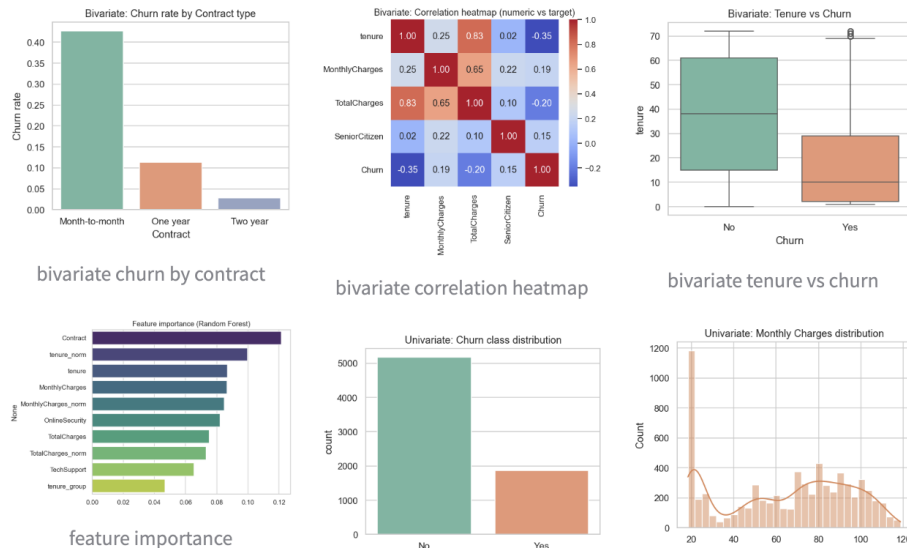


Fig 7. Univariate and bivariate EDA visualizations (saved automatically to reports/figures/).

1.5 DataOps

Implemented in `src/data_pipeline/scheduler.py` using `APScheduler`. The scheduler automates steps 1.3 (pre-processing) and 1.4 (EDA) inside a single pipeline run (`run_pipeline.run_once()`) and triggers it automatically every 2 minutes. Every run logs its start time, each processing step, duration and final SUCCESS/FAILED status to `logs/pipeline.log` and `logs/scheduler.log`. These logs are displayed live on the "DataOps (Scheduler)" tab of the cloud dashboard, which is also capable of auto-refreshing every 10 seconds.

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

Overview  Data Pipeline  EDA  **DataOps (Scheduler)**  ML Pipeline & MLOps  API Access

DataOps — Automated Pipeline Scheduler

A background scheduler (`src/data_pipeline/scheduler.py`) re-runs the pre-processing + EDA steps **every 2 minutes**, logging start/finish time, duration, row counts, and status to:

- `logs/pipeline.log`
- `logs/scheduler.log`

Start it with:

```
python -m src.data_pipeline.scheduler
```

☐ Auto-refresh every 10s

scheduler.log (latest entries)

```
2026-07-10 21:32:41 | INFO | scheduler | APScheduler started. Data pi
2026-07-10 21:32:41 | INFO | scheduler | Scheduler triggering data pi
2026-07-10 21:32:42 | INFO | scheduler | Scheduler run result: SUCCES
```

Fig 8. DataOps tab - scheduler configuration and log viewer.

DataOps — Automated Pipeline Scheduler

A background scheduler (`src/data_pipeline/scheduler.py`) re-runs the pre-processing + EDA steps **every 2 minutes**, logging start/finish time, duration, row counts, and status to:

- `logs/pipeline.log`
- `logs/scheduler.log`

Start it with:

```
python -m src.data_pipeline.scheduler
```

☐ Auto-refresh every 10s

scheduler.log (latest entries)

```
2026-07-10 21:32:41 | INFO | scheduler | APScheduler started. Data pi
2026-07-10 21:32:41 | INFO | scheduler | Scheduler triggering data pi
2026-07-10 21:32:42 | INFO | scheduler | Scheduler run result: SUCCES
2026-07-10 21:39:37 | INFO | scheduler | APScheduler started. Data pi
2026-07-10 21:39:37 | INFO | scheduler | Scheduler triggering data pi
2026-07-10 21:39:38 | INFO | scheduler | Scheduler run result: SUCCES
2026-07-10 21:41:37 | INFO | scheduler | Scheduler triggering data pi
2026-07-10 21:41:38 | INFO | scheduler | Scheduler run result: SUCCES
```

pipeline.log (latest entries)

```
2026-07-10 21:12:10 | INFO | eda | Saved figure: /Users/SEKARGO/Docum
2026-07-10 21:12:10 | INFO | eda | === EDA finished; summary saved to
2026-07-10 21:12:10 | INFO | run_pipeline | ##### DATA PIPELINE RUN
```

Fig 9. scheduler.log proving 3 consecutive automated runs at 21:39:37 and 21:41:37 - exactly 2 minutes apart, each SUCCESS.

Sub-Objective 2: Design and Development of a Machine Learning Pipeline (Weight: 5 marks)

2.1 Model Preparation

Two classification algorithms were selected for the churn prediction problem: Logistic Regression (a strong, interpretable linear baseline for binary classification) and Random Forest (an ensemble tree-based model that captures non-linear feature interactions). Both are implemented in `src/ml_pipeline/train.py` using scikit-learn.

2.2 Model Training

The processed dataset (7,043 rows x 23 features after encoding) was split into a 70% training set and a 30% testing set using a stratified split (`train_test_split`, `test_size=0.30`, `stratify=y`) to preserve the churn/no-churn class ratio. Both models were then trained on the training split.

2.3 Model Evaluation

Models were evaluated on the held-out 30% test set using 5 metrics: Accuracy, Precision, Recall, F1-score and ROC-AUC.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8041	0.6503	0.5668	0.6057	0.8429
Random Forest	0.7956	0.6511	0.4955	0.5628	0.8344

Best model by F1-score: Logistic Regression.

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

[view](#) [🔧 Data Pipeline](#) [🔍 EDA](#) [🕒 DataOps \(Scheduler\)](#) [🏠 ML Pipeline & MLOps](#) [🔑 API Access](#)

ML Pipeline & MLOps

- **Model preparation:** Logistic Regression & Random Forest (2 algorithms)
- **Training:** 70% train / 30% test split
- **Evaluation:** Accuracy, Precision, Recall, F1-score, ROC-AUC
- **MLOps:** every run's params/metrics/model are tracked with **MLflow** (`mlruns/` folder) - run `mlflow ui --port 5000` to browse the experiment tracking UI.

Model comparison

model	accuracy	precision	recall	f1_score	roc_auc	train_duration_sec
Logistic Regression	0.8041	0.6503	0.5668	0.6057	0.8429	1.06
Random Forest	0.7956	0.6511	0.4955	0.5628	0.8344	0.435

🏆 Best model by F1-score: **Logistic Regression**

Confusion matrices

Fig 10. Model comparison table on the dashboard.

- **Training:** 70% train / 30% test split
- **Evaluation:** Accuracy, Precision, Recall, F1-score, ROC-AUC
- **MLOps:** every run's params/metrics/model are tracked with **MLflow** (`mlruns/` folder) -
run `mlflow ui --port 5000` to browse the experiment tracking UI.

Model comparison

model	accuracy	precision	recall	f1_score	roc_auc	train_duration_sec
Logistic Regression	0.8041	0.6503	0.5668	0.6057	0.8429	1.06
Random Forest	0.7956	0.6511	0.4955	0.5628	0.8344	0.435

🏆 Best model by F1-score: **Logistic Regression**

Confusion matrices

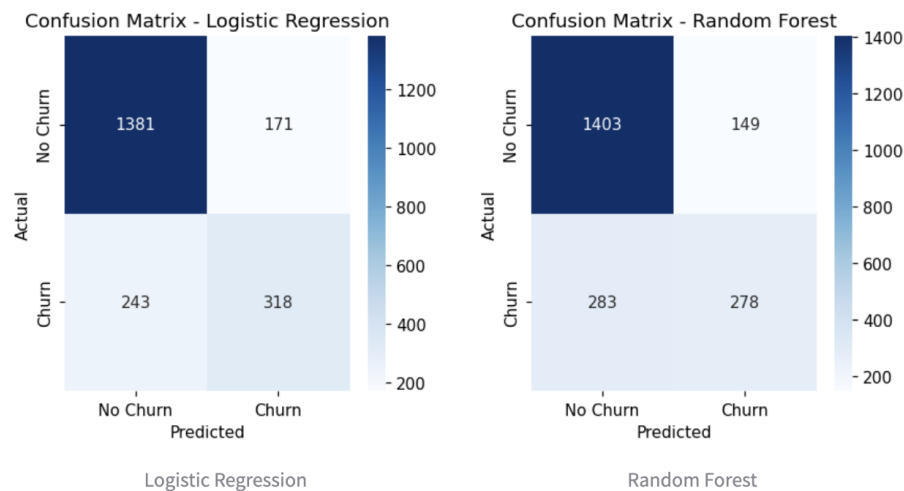


Fig 11. Confusion matrices for both models on the test set.

2.4 MLOps

MLflow (`mlruns/` local tracking store) is used as the MLOps layer. For every training run, the pipeline logs: parameters (model_type, train/test row counts, number of features), 4+ metrics (accuracy, precision, recall, F1-score - plus ROC-AUC and training duration), the trained model artifact, and the confusion-matrix image as an artifact. Each model is also registered in the MLflow Model Registry (telco-churn-logistic-regression, telco-churn-random-forest), enabling versioned deployment tracking. The MLflow Tracking UI (`mlflow ui` / `mlflow server`) provides live monitoring of all runs and their metrics.

mlflow2.14.2

ExperimentsModels

GitHubDocs

Experiments

+

◀

Search Experiments

☒

Default

☐

telco-churn-prediction

Default

Provide Feedback

Add Description

Share

Q metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

Datasets

Sort: Created

Columns

Group by

Table

Chart

Evaluation

Experimental

Traces

Experiment

☐	Run Name	Created		Date
--------------------------	----------	---------	--	------

No runs logged

No runs have been logged yet. [Learn more](#) about how to create ML model training runs in this experiment.

0 matching runs

Fig 12. MLflow Tracking UI - experiment list.

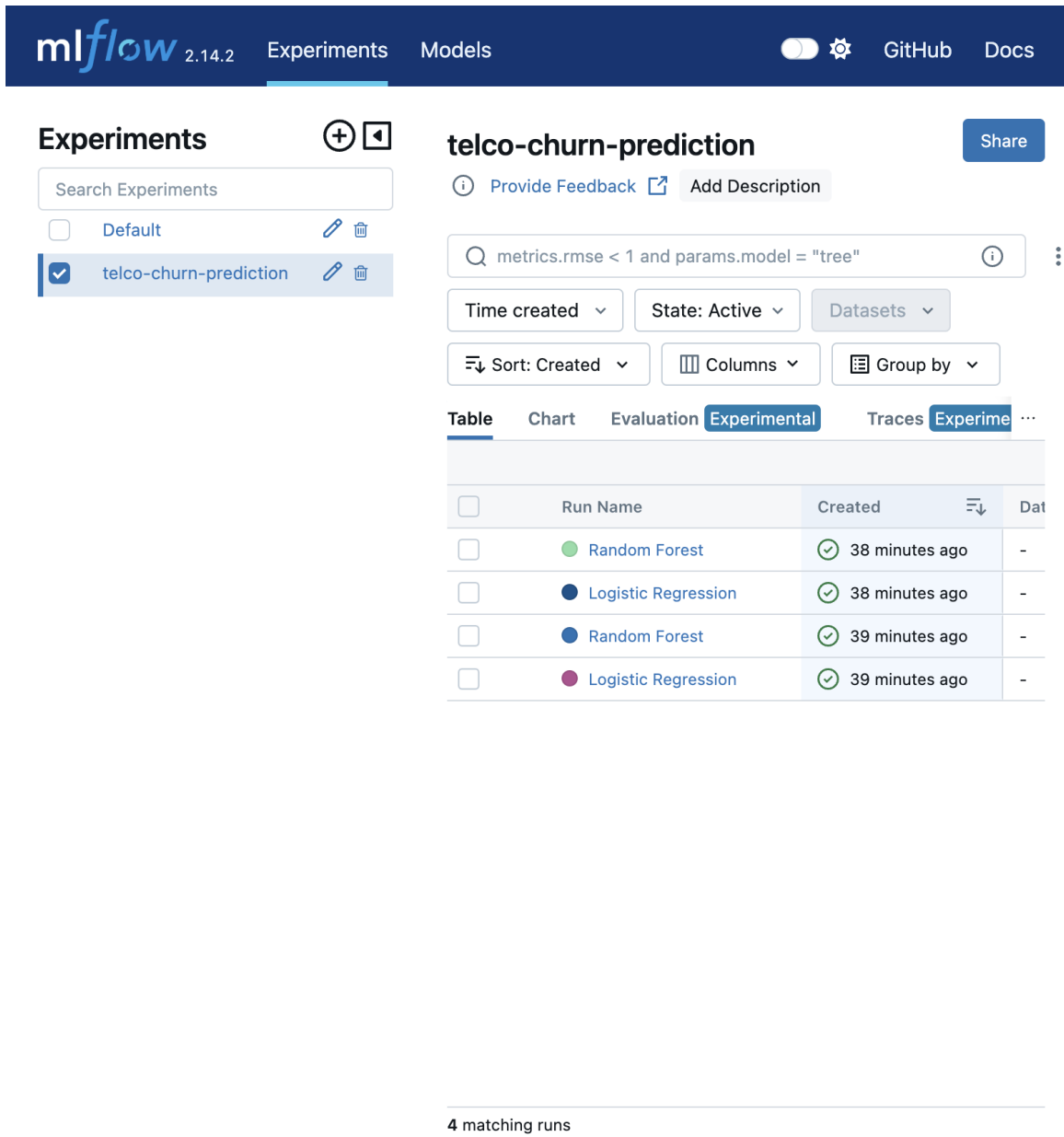


Fig 13. MLflow Tracking UI - 4 logged runs for the telco-churn-prediction experiment (2 models x 2 pipeline executions).

Sub-Objective 3: API Access (Weight: 2 marks)

3.1 Retrieve Key Application Details

Instead of writing a custom REST API, this project uses the built-in REST API that the MLflow Tracking Server exposes automatically (src/api/fetch_app_details.py). The MLflow server is started with:

```
mlflow server --backend-store-uri file:./mlruns --host 127.0.0.1 --port 5000
```

The module calls the following built-in endpoints to retrieve application/deployment details:

- GET /api/2.0/mlflow/experiments/get-by-name - experiment metadata (the application's data-science "project").
- POST /api/2.0/mlflow/runs/search - pipeline run / flow details (status, start/end time, params, metrics).
- GET /api/2.0/mlflow/registered-models/search - registered model (deployment) information.
- GET /api/2.0/mlflow/model-versions/search - model version details (deployment stage, artifact source).

3.2 Display Application Details

The "API Access" tab of the cloud dashboard calls these APIs live (via a button click) and renders at least 4 application details on screen:

- 1) Experiment details - experiment_id, name, artifact_location, lifecycle_stage, creation_time.
- 2) Latest pipeline runs (flow details) - run_uuid, run_name, status, start/end time, artifact_uri, logged params & metrics.
- 3) Registered models (deployment info) - model name, creation/update timestamps, latest version.
- 4) Model versions - version number, current_stage, source path, MLflow model size and serialization format.

Telco Customer Churn — API-Driven Cloud Native ML Application

Business problem: predict which telecom customers are likely to churn, so retention teams can act proactively. Dataset: IBM/Kaggle Telco Customer Churn (7,043 customers, 21 attributes).

[view](#) [Data Pipeline](#) [EDA](#) [DataOps \(Scheduler\)](#) [ML Pipeline & MLOps](#) [API Access](#)

API Access — MLflow built-in REST API

This tab calls the **built-in MLflow tracking-server REST API** (no custom API code) to retrieve application / deployment details.

Start the tracking server first:

```
mlflow server --backend-store-uri file:./mlruns --host 127.0.0.1 --port
```

MLflow tracking server URL

`http://127.0.0.1:5000`

Fetch application details via API

1 Experiment details

```
{
  "experiment_id" : "498093849882541941"
  "name" : "telco-churn-prediction"
```

Fig 14. API Access tab - Detail #1: Experiment details retrieved via the MLflow REST API.

fetch application details via API

1 Experiment details

```
{
  "experiment_id" : "498093849882541941"
  "name" : "telco-churn-prediction"
  "artifact_location" :
    "file:///Users/SEKARGO/Documents/Learn/bits/CourseMaterials/SEM 3/API
    driven/Assignment1/mlruns/498093849882541941"
  "lifecycle_stage" : "active"
  "last_update_time" : 1783698466957
  "creation_time" : 1783698466957
}
```

2 Latest pipeline runs (flow details)

```
[
  0 : {
    "info" : {
      "run_uuid" : "3061bfa33f504c26a3a6729a6ffefa2d"
      "experiment_id" : "498093849882541941"
      "run_name" : "Random Forest"
      "user_id" : "SEKARGO"
      "status" : "FINISHED"
      "start_time" : 1783698498022
      "end_time" : 1783698501249
      "artifact_uri" :
        "file:///Users/SEKARGO/Documents/Learn/bits/CourseMaterials/SEM 3/API
        driven/Assignment1/mlruns/498093849882541941/3061bfa33f504c26a3a6729a6ffef
        "lifecycle_stage" : "active"
    }
  }
]
```

Fig 15. API Access tab - Detail #2: Latest pipeline run (flow) details.

Deploy ⋮

```
    "serialization_format": "cloudpickle", "code": null}},
    "model_uuid": "7c9ef2c131c0475385fc34abb58629e3",
    "mlflow_version": "2.14.3", "model_size_bytes":
    1236}]"
  }
  4 : {
    "key" : "mlflow.source.type"
    "value" : "LOCAL"
  }
]
}
"inputs" : {}
}
```

3 Registered models (deployment info)

```
  [
    0 : {
      "name" : "telco-churn-logistic-regression"
      "creation_timestamp" : 1783698470658
      "last_updated_timestamp" : 1783698498014
      "latest_versions" : [
        0 : {
          "name" : "telco-churn-logistic-regression"
          "version" : "2"
          "creation_timestamp" : 1783698498014
          "last_updated_timestamp" : 1783698498014
          "current_stage" : "None"
          "source" :
            "file:///Users/SEKARGO/Documents/Learn/bits/CourseMaterials/SEM 3/API
```

Fig 16. API Access tab - Detail #3: Registered models (deployment info).

Tools & Technologies Used

- Python 3.12, pandas, numpy, scikit-learn - data processing & ML
- matplotlib, seaborn - EDA visualizations
- APScheduler - DataOps 2-minute pipeline scheduling
- MLflow - MLOps experiment tracking, model registry, built-in REST API
- Streamlit - interactive cloud dashboard (deployable to Streamlit Community Cloud)
- Kaggle / IBM Telco Customer Churn dataset

How to Run This Project

Full setup, dataset-download fallback instructions, and step-by-step run/deploy commands are documented in README.md at the root of this project. In summary:

- 1. `python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt`
- 2. `python -m src.data_pipeline.run_pipeline` (one-off pipeline run)
- 3. `python -m src.data_pipeline.scheduler` (DataOps, every 2 min)
- 4. `python -m src.ml_pipeline.train` (ML pipeline + MLOps)
- 5. `mlflow server --backend-store-uri file:./mlruns --port 5000` (API access)
- 6. `streamlit run src/dashboard/app.py` (cloud dashboard)

Conclusion

This project delivers an end-to-end, API-driven, cloud-native data-science application for telecom customer churn prediction. It automates data ingestion, cleaning and EDA through a scheduled DataOps pipeline; trains, evaluates and tracks two ML models through an MLOps layer; and exposes key application/deployment details through a built-in REST API - all surfaced through a single interactive Streamlit cloud dashboard.